Name: P H Devesh

Register Number: 1911260236

Subject: Python Programming (CSA0805)

Assignment: CO2

Q1 – Decisions with Many Boundaries

Decision rule

Use a mutually exclusive if–elif–else chain: up to 100 units receives full subsidy; 101–200 receives partial subsidy only with a ration card; above 200 receives the senior exception only when a member is older than 80; otherwise no subsidy. Separate if statements can independently execute more than one branch and issue conflicting decisions.

Boundary predictions

100 → full; 101 with ration card → partial; 101 without card → none; 200 with card → partial; 200 without card → none; 201 with age over 80 → senior exception; 201 without age exception → none. The predicates must use inclusive bounds where specified.

Design choice and alternative

A flat if–elif chain is clearest because the slabs are ordered and exclusive. Nested if statements can obscure the complete rule; combined Boolean expressions are concise when parenthesised. A match–case can match guarded conditions, while a list of upper-limit rules can be scanned in order. For k slabs, a chain or list scan is O(k) worst case and O(1) extra space; a small fixed set of branches is effectively constant. A dictionary lookup is not ideal for continuous ranges.

Complexity

The decision uses a fixed number of comparisons, O(1) time and O(1) space for the stated four bands.

Python Program

```
def subsidy(units, ration_card, age_over_80):
    if units <= 100:
        return "full subsidy"
    elif 101 <= units <= 200 and ration_card:
        return "partial subsidy"
    elif units > 200 and age_over_80:
        return "senior-citizen exception"
    return "no subsidy"

cases = [(100, False, False), (101, True, False), (200, True, False),
         (201, False, True), (201, False, False), (100, True, True),
         (150, False, True), (200, True, True)]
for case in cases:
    print(case, "->", subsidy(*case))
```